Embedding Live Machine Learning in Csound: Emotion-Driven Harmony via an ONNX-Runtime Opcode

Jinlan Zhao, Li (Charlie) Shi, and Richard Boulanger

Berklee College of Music, Boston, MA, USA
{jzhao9, cshi, rboulanger}@berklee.edu

Abstract. Harmonic generation in Csound typically requires explicit chord specification in the score or coordination with an external process. We present the *Affective Harmony Machine* (AHM), a system that embeds an Open Neural Network Exchange (ONNX) format machine-learning model as a native Csound opcode, demonstrated through affect-driven chord progression generation. The system trains a Random Forest classifier on 108 annotated jazz and pop chord progressions (six emotion classes, seven scale/mode categories) and exports it to ONNX. We also present `emoChord`, a Csound opcode that accepts an emotion string, performs live ML inference at i-rate via the ONNX Runtime C API, and schedules harmonically appropriate chord events into a synthesis instrument. No Python interpreter or external process is required at runtime; the current release targets macOS (arm64 natively, x86_64 via Rosetta). The implementation demonstrates a general pattern for embedding ONNX-compatible classifiers as native Csound plugins.

Keywords: Csound opcode, ONNX Runtime, machine learning, harmony generation, emotion, chord progression, affective computing

1 Introduction

Csound [7] has a long tradition of algorithmic composition. John ffitch's `csbeats` pre-processor [9] showed that common-practice notation, including pitches, rhythms, and dynamics, can map directly to Csound score events, and McCurdy's Haiku orchestras demonstrate elegant algorithmic chord generation within Csound. Yet in most of these systems, harmony remains the composer's explicit responsibility: chords are either written into the score by hand or drawn from probability tables constructed in advance. Shi and Boulanger's CStore [1] demonstrated AI-driven orchestra code from natural language, though as an external process, sharpening the question: how far can machine intelligence be embedded *within* Csound itself?

We answer concretely: any ONNX-compatible classifier following the same input/output tensor contract can be embedded as a native Csound i-rate opcode via the ONNX Runtime C API with no external-process dependency. As a demonstration, the *Affective Harmony Machine* (AHM) targets affect-driven harmony, where emotional states map to musical parameters such as mode and perceived affect [2]: a composer writes `i1 0 4 "joyful"` to receive chord events, i.e., MIDI-pitched notes inserted at i-rate via Csound's score API (Section 4.2) into any synthesis instrument.

2 Dataset

The system is trained on two curated harmony corpora. The jazz dataset covers functional harmony, including secondary dominants, tritone substitutions, and deceptive resolutions, across 25 unique progressions, each realized in four jazz voicing styles (Four-Way Close, Drop 2, Drop 3, Drop 2+4); the four variants share the same progression class label; voicing style is an auxiliary label for a co-trained jazz voicing classifier (predicting which of the four arrangements to apply to a given progression), not part of the 108-class progression target. All progressions and voicing assignments were hand-coded by the authors.[1] The pop dataset covers diatonic, modal, and borrowed-chord idioms across 83 unique progressions. Table 1 summarizes the combined dataset.

Each row is annotated along three orthogonal dimensions: **Emotion** (6 classes: Joyful, Vital, Epic, Uneasiness, Depressive, Despair), **Scale/Mode** (7 classes: Ionian, Aeolian, Dorian, Phrygian, Lydian, Mixolydian, Harmonic minor), and **Key** (12 pitch centers; the major/minor distinction is omitted since mode is already captured by the Scale/Mode label). Progressions are encoded in Roman numeral notation

[1] `https://github.com/JinlanZhao/AHM-EmoChord`

Table 1. Dataset summary after preprocessing.

Source	Progressions	Rows	Coverage
Jazz	25	1,200	12 keys × 4 voicings
Pop	83	996	12 keys
Combined	**108**	**2,196**	–

(*e.g.,* `I-V-vi-IV`), making the representation key-invariant: the model learns harmonic character, not absolute pitch. Pop progressions were expanded to all 12 keys via automated transposition. Emotions are mapped approximately onto Hevner's adjective circle [2]: Joyful and Vital occupy the bright/vivacious quadrant; Epic occupies majestic/triumphant; Uneasiness occupies restless/yearning; and Depressive and Despair occupy melancholic/grief.

3 Machine Learning Pipeline

3.1 Feature Design

The classification task is: *given (emotion, scale), predict the most probable chord progression type*, a 2-input, 108-class problem. Key is intentionally excluded from the feature vector because transposition does not alter harmonic character, and including it would fragment the training data into 12 identical shards. The dataset spans 23 of the 42 possible `(emotion, scale)` combinations (6 emotions × 7 scales); the model learns a distinct probability distribution over the 108 classes for each observed pair, enabling temperature-weighted sampling (Section 4.3) rather than always returning the single most-probable progression.

3.2 Random Forest with Optuna Tuning

A Random Forest Classifier [3] is chosen because its `predict_proba()` output provides a per-class probability vector over all 108 progression types, directly usable as a stochastic sampling distribution at runtime; the downstream temperature scaling and weighted sampling (Section 4.3) act on this vector to enable probabilistic affective selection.

Within each observed (emotion, scale) cell all training feature vectors are identical, so the RF's vote-averaging mechanism converges to the empirical class frequency distribution for those 23 cells; a per-cell lookup table of empirical counts would approximate the same `predict_proba` output. The RF is adopted over a plain lookup for three reasons: (1) `skl2onnx` exports it directly to the fixed tensor contract that `emoChord` expects, with no additional serialization tooling; (2) for the 19 unobserved (emotion, scale) cells, the RF routes inputs to the nearest trained leaf rather than failing outright; and (3) the ONNX tensor contract is model-agnostic: any classifier producing the same $[N, 108]$ float32 output can replace the RF without modifying the plugin.

Hyperparameters are optimized with Optuna [4] (50 trials, 5-fold cross-validation, the training script co-tunes a jazz voicing classifier, but only the progression model is exported). Training follows a two-stage procedure: hyperparameter search on an 80/20 stratified split, then retraining on the complete 2,196-row dataset. Retraining is critical: stratification by emotion (6 classes) rather than the 108-class target means rare progressions absent from the 80% fold would break index mappings at runtime.

3.3 ONNX Export

The trained classifier is serialized to a self-contained binary file (`gen_model.onnx`) using `skl2onnx` [5]. Two export options are critical for C interoperability: `zipmap=False` produces a plain numeric array rather than a Python dictionary, enabling direct C array access; `target_opset=18` pins the operator-set version to the one bundled with the plugin. A companion lookup table `gen_data.tsv` (108 progressions × 12 pitch classes = 1,296 rows) maps each progression index to a human-readable chord-name string per key (*e.g.,* `Cm7-F7-Bbmaj7`); jazz voicing variants share the same progression index, and major/minor spellings are resolved according to the sampled scale.

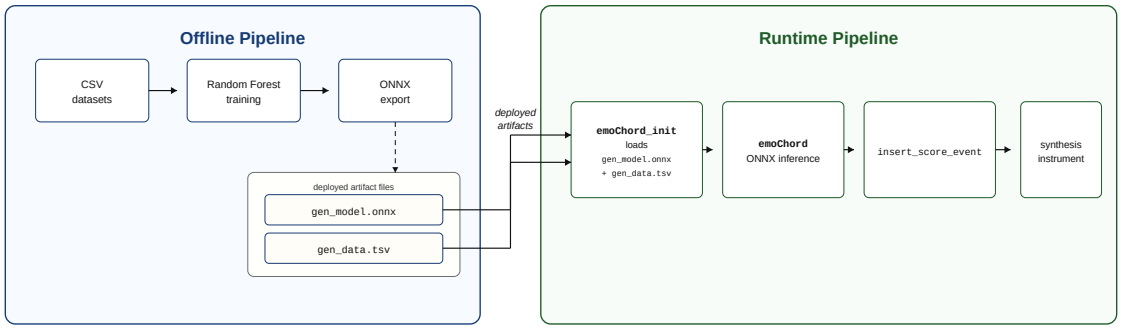


Fig. 1. AHM two-phase architecture. Offline (left): training and ONNX export of `gen_model.onnx` and `gen_data.tsv`. Runtime (right): native Csound inference and chord event scheduling via `emoChord`.

4 Embedding ML in a Csound Opcode

4.1 Overview

The authors built the plugin as a `.dylib` that links directly against the ONNX Runtime C library [6]; `emoChord_init SModelPath, SDataPath` loads the ONNX model and vocabulary table once at i-rate, storing all shared state in static C globals available to every subsequent `emoChord` call. **Known limitation:** this design is not thread-safe across concurrent Csound instances. Figure 1 shows the two-phase architecture.

Inference runs at i-rate with no external process needed; on Apple Silicon (arm64), median single-sample inference is 14 μs, far below typical score-event scheduling timescales, and model load is 52 ms.

4.2 Live Inference Pipeline in `emoChord`

`emoChord SEmotion, iInstr, iStart, iDur, iAmp [, iTemp, iOctave]` executes the following seven-step pipeline entirely inside the Csound process:

1. **Resolve emotion**: scan `DataEntry[]` for rows matching `SEmotion` (case-insensitive) to obtain `emotion_id`.
2. **Sample scale**: collect all `scale_id` values for that emotion, weighted by dataset frequency; draw one by temperature-weighted inverse-CDF.
3. **ONNX inference**: call `OrtApi->Run()` with input $[1, 2] = \{emotion_id, scale_id\}$, retrieving $\mathbf{p} \in \mathbb{R}^{108}$.
4. **Filter**: mask $\mathbf{p}$ to `prog_id` values present for the sampled pair.
5. **Temperature scaling**: apply $p'_i = p_i^{1/T}$ and renormalize over the filtered candidates.
6. **Sample progression**: draw one `prog_id` by weighted inverse-CDF.
7. **Schedule chords**: look up the `chord_name` string (*e.g.,* `Cm7-F7-Bbmaj7`), parse tokens into MIDI note numbers, and call `insert_score_event()` on instrument `iInstr`.

4.3 Temperature as Compositional Control

The `iTemp` parameter (default 1.0) translates the ML temperature concept into a direct musical control. At low T, sampling concentrates on the highest-probability progression; at $T = 1$ it follows the model's trained distribution; at $T > 1$ the distribution flattens toward uniform, increasing harmonic variety. A composer can place two adjacent score events with the same emotion string but different temperatures: one to anchor a phrase and one to introduce a surprise chord, without touching the model or any other instrument parameter.

4.4 Usage Example

A minimal orchestra defines two instruments following the standard Csound idiom [8,9]: `instr 1` accepts an emotion string as a score p-field and invokes `emoChord` (initialized at the orchestra header via `emoChord_init`), which performs inference and schedules one score event per chord tone into `instr 2` (`p4` = MIDI note number, `p5` = amplitude); `instr 2` is any synthesis voice of the composer's choice. At each event, `emoChord` prints the full reasoning chain (*e.g.,* `[emoChord] "joyful" -> Ionian -> C-G-Am-F`), making ML decisions inspectable. Figure 2 shows the console output for three score events. A complete `.csd` with an FM synthesis voice is available in the repository.[2]

```
new alloc for instr 1:
emoChord [joyful]: Gm7-C7-Fmaj7
new alloc for instr 2:
new alloc for instr 2:
new alloc for instr 2:
new alloc for instr 2:
  rtevent:      T  1.000 TT  1.000 M:  0.73339
0.73339
  rtevent:      T  2.000 TT  2.000 M:  0.74485
0.74485
B  0.000 ..  6.000 T  6.000 TT  6.000 M:  0.72180
0.72180
emoChord [depressive]: Am-Bm-Em
  rtevent:      T  7.000 TT  7.000 M:  0.57338
0.57338
  rtevent:      T  8.000 TT  8.000 M:  0.57727
0.57727
B  6.000 .. 18.000 T 18.000 TT 18.000 M:  0.57611
0.57611
emoChord [uneasiness]: Ab-Db-Cm-F
  rtevent:      T 19.000 TT 19.000 M:  0.57931
0.57931
  rtevent:      T 20.000 TT 20.000 M:  0.58278
0.58278
```

Fig. 2. `emoChord` console output for three score events.

5 Results and Conclusion

After Optuna tuning (50 trials, 5-fold cross-validation), the model achieves held-out top-1 accuracy of 24.8% and top-3 of 49.1%, against a uniform random baseline of 0.9% (1/108) and a majority-class-per-pair baseline of 24.8%; the accuracy ceiling is a design property: since feature vectors within a cell are identical, top-1 accuracy cannot exceed the dominant-class proportion per cell.

The mode selections produced by the system align with Hevner's established mode-affect associations [2]: by annotation design, joyful and vital progressions draw predominantly Ionian (major) scales, consistent with Hevner's bright/happy adjective cluster, while depressive and despair classes draw predominantly Aeolian and Harmonic minor (melancholic/sad), and uneasiness draws Dorian and Phrygian (restless/yearning).

The dataset built for this work, combining 2,196 rows of jazz and pop progressions under a unified six-class affect taxonomy, is itself a contribution we hope will prove useful to others. Beyond the dataset, this work establishes a reusable pattern for native ML in Csound: any ONNX classifier with a compatible tensor contract can replace the current model without touching the plugin.

6 Future Directions

Planned technical extensions include a k-rate (per-control-cycle) variant for dynamic emotion changes during performance, an `iKey` parameter for key-locked output, longer sequence generation, and Windows/Linux porting. On the research side, we plan to add a genre input to resolve jazz/pop ambiguity, standardize emotion labels against an established affect taxonomy, and conduct a formal perceptual evaluation. The current fixed emotion taxonomy is also a known limitation: emotion is inherently subjective and perceived differently across listeners, so future work will expand the dataset to reflect this diversity and introduce a natural-language interface through which users can describe their intended affect in their own words.

[2] `https://github.com/JinlanZhao/AHM-EmoChord`

References

1. Shi, L.C., Boulanger, R.: CStore: A Framework for Generating Editable, Versionable Csound Orchestra Code – From Text to Sound and Music. In: Proceedings of the 2026 IEEE Conference on Artificial Intelligence (CAI), pp. 237–243. IEEE, Granada (2026)
2. Hevner, K.: The affective character of the major and minor modes in music. American Journal of Psychology 47, 103–118 (1935)
3. Pedregosa, F. et al.: Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research 12, 2825–2830 (2011)
4. Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M.: Optuna: A next-generation hyperparameter optimization framework. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 2623–2631 (2019)
5. Dupré, X. et al.: sklearn-onnx: Convert scikit-learn models to ONNX. `https://onnx.ai/sklearn-onnx/` (2019–present)
6. ONNX Runtime Developers: ONNX Runtime inference engine. `https://onnxruntime.ai` (2018–present)
7. Lazzarini, V. et al.: Csound: A Sound and Music Computing System. Springer, Cham (2016)
8. Boulanger, R., Lazzarini, V. (eds.): The Audio Programming Book. MIT Press, Cambridge (2011)
9. Boulanger, R. (ed.): The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press, Cambridge (2000)